

Privacy Leakage and Lightweight Defenses in Graph Neural Networks for Sensitive Data

Krish Sharma

Academy of Math, Science, and Applied Technology
Ed W. Clark High School
Las Vegas, NV, USA
krish.sharma@unlv.edu

Shaikh Arifuzzaman

Department of Computer Science
University of Nevada, Las Vegas
Las Vegas, NV, USA
shaikh.arifuzzaman@unlv.edu

Abstract—Graph neural networks (GNNs) are emerging in domains such as healthcare, finance, and cyber-physical systems, where graph structure is useful but privacy risk must be measured carefully. This paper studies supervised node-membership inference: given black-box access to a model’s output probabilities, an adversary tries to infer whether a node’s label was included in the supervised training mask used to optimize the model. To support structure-aware evaluation, we introduce a controlled benchmark that varies graph homophily and edge density independently. We compare GCN and GraphSAGE with feature-only baselines, including logistic regression and MLP, and evaluate predictive utility, calibration, and membership leakage under lightweight score-based attacks: a confidence-feature attack, a true-label threshold attack, and a shadow-model transfer attack. The results show that leakage is not uniformly higher for GNNs. Instead, it depends on graph structure, architecture, feature separability, and posterior behavior. In controlled low-homophily synthetic regimes, GCN leakage rises; in Cora and Citeseer, feature-only baselines can leak more than graph models. Label smoothing often increases attack AUROC, but its calibration effects are dataset-dependent. Edge sparsification, implemented as permanent undirected edge thinning for both training and inference, has mixed effects rather than a universal privacy benefit. Overall, the benchmark supports reproducible evaluation of utility–privacy tradeoffs for supervised node classification under lightweight membership attacks.

Index Terms—graph neural networks, membership inference, privacy, homophily, graph density, lightweight defenses, calibration, empirical evaluation, shadow models

I. INTRODUCTION

Graph neural networks (GNNs) have drawn attention as a standard approach to learn node-level representations from relational data in domains as diverse as healthcare, finance, and communication networks. In the context of transductive node classification, it is necessary to state the meaning of membership in the ground truth labels. The graph, node features, and edges are available to the GNN message passing process even for the nodes whose labels are to be learned; the membership bit simply asks whether the node’s label belongs to the supervised training mask. Membership inference attacks (MIAs) aim to determine whether the membership of training examples in the supervised training set can be inferred from the GNN’s output [1]. As node-level representations are learned through message passing in which the nodes’ representations are coupled with those of their neighbors in

the graph, privacy leakage is also likely to depend on the structure of the graph in which the nodes reside [5], [7]. Unlike in a typical machine learning problem in which the training examples are assumed to be independent and identically distributed, the nodes in a graph share edges. Consequently, the classical assumption underlying the optimization process that the training examples appear independently in the loss function is violated by the graph structure of the nodes, invalidating some generalization and privacy analyses that are possible for classical machine learning problems.

We introduce an empirical benchmark to answer four research questions. **RQ1** aims to answer whether graph neural networks are more vulnerable to membership inference attacks than non-graph models with similar feature dimensionality and accuracy. **RQ2** investigates whether homophily and edge density in graphs impact the success of membership inference attacks, both of which can be controlled during the construction of the graph. **RQ3** aims to investigate whether any defenses to membership inference attacks can be implemented at the cost of minimal loss in model accuracy. Finally, **RQ4** seeks to understand the tradeoffs between utility and privacy for each of the models, defenses, and regimes of graphs investigated in this paper. Together, these research questions enable us to not just determine whether membership information of a model is leaked, but also why it is leaked, and what interventions can mitigate that leakage.

Contributions. We present a framework for conducting experiments using YAML files. We conduct experiments on two-layer graph convolutional networks and GraphSAGE using logistic regression and multilayer perceptron as baselines on six synthetic datasets as well as Cora and Citeseer. For the experiments we use three attack methods based on scores with well-defined score features as well as five defenses with fixed hyperparameters. For each experiment we report the mean and standard deviation of the results across multiple random seeds, perform significance tests on the results to examine whether the differences between the results are statistically significant, and report the expected calibration error as a measure of the calibration of the models. Our results include findings on the structure and model-dependent nature of the risk of membership in supervised learning models, the failure modes of label smoothing as a means of achieving privacy, and

the impact of permanent edge sparsification that is setting-dependent rather than universal in its privacy benefits. Additionally, we provide detailed descriptions of the methods used in the paper including the way in which the data was generated, how the data was split into training, validation, and test sets, and the steps that were taken to ensure the reproducibility of the results.

By isolating factors that are normally confounded in a single real-world graph, such benchmark graphs enable us to study the effect of each of these factors in isolation. Each of these findings is thus limited to the settings of our generator family; further experiments on real-world graphs are required to establish the external validity of these findings, but such experiments are enabled by our codebase and citation-network loaders.

Evaluation metrics combine predictive utility (accuracy, macro F1, macro AUROC), probabilistic calibration [8], and three membership-risk summaries aligned with prior literature [1], [2], [16]. Reporting calibration alongside attacks is critical because many defenses reshape posteriors without reducing distinguishability under membership tests [3], [17].

II. RELATED WORK

A. Membership inference on classical models

Foundational work used shadow models and attack classifiers based on statistics of the model’s outputs [1]. Subsequent work relaxed assumptions about the knowledge of the model’s architecture and the distribution of its training data [2]. Theoretical work investigated the relationship between the advantage of membership inference attacks and concepts related to the model’s generalization and loss functions [3]. Subsequent work considered the case where the attacker has access to the model’s architecture and training data [15], [16]. More recent work considers evaluating the attacks at a given false positive rate [4]. We evaluate the attacks using the area under the ROC curve, though we plan to utilize stricter metrics in future work.

B. Membership inference on graphs

The study of node-level membership inference for GNNs reveals insights from both posterior-based and shadow-model attackers [5]. For the graph-level setting, different drivers of model leakage have been identified [6]. Furthermore, surveys of the literature have categorized graph neural network attacks according to their purpose, grouping attacks like membership inference with model reconstruction and attribute inference [7]. Related work on graph privacy further motivates the study of membership inference for graph neural networks [29], [30].

C. Architectures and lightweight defenses

Graph convolution [8] and neighborhood aggregation in GraphSAGE [9] define our hypothesis classes, with broader GNN context from graph attention and expressivity analyses [23], [24]. DropEdge [10] perturbs topology during training. Output-centric defenses interact with label-only adversaries

[11]. Differential privacy for GNNs provides rigorous guarantees with higher complexity [12]; our non-DP baselines should therefore be interpreted relative to the wider DP literature [19]–[21].

III. PRELIMINARY CONCEPTS FOR PRIVACY-ORIENTED GRAPH LEARNING

A. Homophily, heterophily, and learning difficulty

Homophily is a factor that describes how likely it is for edges to connect nodes that have the same class label. High levels of homophily are seen in examples such as social networks or citation networks, where nodes with the same labels tend to be connected by edges. Heterophily represents the opposite of this concept, where edges tend to exist between nodes with different class labels, leading to issues in models like the Graph Convolutional Network (GCN) that act as low-pass filters for those networks [8]. The graph generator that we propose allows for a tradeoff between the number of edges that connect nodes with the same labels versus those with different labels, which ensures that the homophily among the generated graphs is within the target range (Section VI). Thus, this concept allows for the separation of these factors from model capacity in ways that a single graph and its associated learning problem does not permit.

B. Membership inference in the graph setting

Membership inference with respect to the graph learning problem can refer to several different concepts altogether: whether the node exists in the graph, whether its features and edges are accessible, whether its label is used for supervised learning, and whether it contributes to the message passing steps of the graph learning algorithm [5]. More specifically, membership with respect to the graph is defined as the requirement that the node belongs to the training mask $\mathcal{T}$ and that its label is utilized for the calculation of the supervised learning loss function. Unlike in settings where IID data is used, the loss of one node in the graph impacts the loss of other nodes that are connected to it through edges. Thus, high accuracy on the graph may be achieved with poor membership privacy, indicating that the model is essentially memorizing the training dataset rather than learning how to generalize to new nodes [3]. The different masks $(\mathcal{T}, \mathcal{S})$ are varied in our study to maintain membership to the training set while performing semi-supervised learning.

Intuitively, the attacker is attempting to determine whether the model and its associated loss function have high confidence in the membership of a node to the training set $\mathcal{T}$, rather than attempting to recover the labels or edges of the graph. The confidence of the model can be measured in various ways that consider both the features of a node and the features of the nodes that are connected to it. Thus, two models that exhibit similar accuracies on supervised learning tasks can differ in the degree to which their training process led to the leakage of membership information.

TABLE I
FACTORS IN THE CONFIGURATION-DRIVEN EXPERIMENT GRID (YAML).
CARDINALITY IS THE CARTESIAN PRODUCT SUBJECT TO BASELINE
CONSTRAINTS (FEATURE-ONLY MODELS ONLY TRAIN UNDER NO
DEFENSE).

Factor	Levels
Synthetic regimes	6
Citation graphs (when enabled)	2
Model families	4 (LogReg, MLP, GCN, GraphSAGE)
GNN defenses	6 (none + five mechanisms)
Baseline defenses	1 (none)
Seeds $ S $	5

C. Factorial structure of the benchmark

We adopt a full factorial skeleton over datasets $\times$ models $\times$ defenses $\times$ random seeds, implemented declaratively so that omitting or adding a dataset requires configuration edits only. Graph models exhaust all defense toggles; non-graph models repeat the no-defense condition. Table I summarizes cardinalities. This design supports additive interpretations of main effects (e.g., homophily level) conditional on architecture, although interactions between structure and architecture are expected *a priori*.

IV. NOTATION AND GRAPH LEARNING PRELIMINARIES

Let $G = (V, E)$ be an undirected graph with $|V| = n$ nodes. We encode edges by a symmetric adjacency matrix $A \in \{0, 1\}^{n \times n}$ or equivalently an edge index tensor listing endpoints row-wise for both directions. Node features are $X \in \mathbb{R}^{n \times d}$ and integer labels $y \in \{0, \dots, C - 1\}^n$. For message passing we use degree matrix $D_{ii} = \sum_j A_{ij}$ and normalized propagation $\tilde{A} = D^{-1/2} A D^{-1/2}$ for graph convolutional layers as in Kipf and Welling [8].

A two-layer graph convolutional network (GCN) computes

$$H^{(1)} = \sigma(\tilde{A} X W^{(0)}), \quad Z = \tilde{A} H^{(1)} W^{(1)}, \quad (1)$$

where σ is ReLU applied row-wise, dropout acts on $H^{(1)}$ during training, and rows of Z are logits. For GraphSAGE with mean aggregation (two layers), each layer applies a learned affine map after neighborhood mean pooling over one-hop neighbors [9]; we denote final logits likewise by Z . Baseline models ignore A and map X directly to logits.

Training masks $m_{tr}, m_{te} \in \{0, 1\}^n$ satisfy $m_{tr} + m_{te} = \mathbf{1}$ entrywise on synthetic benchmarks with a 50/50 split unless stated otherwise. Softmax probabilities are $\hat{p}_v = \text{softmax}(z_v)$.

V. THREAT MODEL AND PRIVACY GOAL

A. Adversary capabilities

We assume a *black-box* adversary who may submit nodes $v \in V$ and receives $\hat{p}_v \in \mathbb{R}^C$, the model’s predicted probability vector after softmax. This oracle abstracts deployed APIs that expose confidence scores or calibrated probabilities. The adversary knows which label y_v is under evaluation (standard for node-level membership games [5]) and may compute derived scalars from $\hat{p}_v$. We do *not* grant access to intermediate embeddings, gradients, or parameters θ .

TABLE II
PRIMARY NOTATION USED THROUGHOUT.

Symbol	Meaning
n, d, C	Nodes, feature dimension, classes
$A, \tilde{A}$	Adjacency and normalized propagation
X, y	Feature matrix and label vector
$z_v, \hat{p}_v$	Logits and softmax probabilities at v
$\mathcal{T}, \mathcal{S}$	Supervised training-mask and held-out node sets
$\phi(\cdot)$	Four-dimensional attack feature map
$h(G), \rho$	Observed homophily, density fraction
ECE	Expected calibration error ($B=10$ bins)
α, r_{de}, r_{es}	Smoothing, DropEdge, sparsification rates

Our threat model is grounded in how graph models are often served in production: clients may receive probability vectors or confidence-ranked outputs even when the implementation is opaque. By restricting the adversary to black-box access to these outputs, we estimate leakage under service-realistic conditions.

B. Membership experiment

Let $\mathcal{T} \subseteq V$ denote nodes whose labels are used in the supervised training loss and let $\mathcal{S} \subseteq V$ denote held-out nodes with $\mathcal{T} \cap \mathcal{S} = \emptyset$. The nodes in both sets may still exist in the transductive graph and may participate in message passing. For a query v , membership inference seeks to distinguish $v \in \mathcal{T}$ versus $v \in \mathcal{S}$ using only $\hat{p}_v$ and y_v . We summarize distinguishability by attack area under the ROC curve (AUROC) and attack accuracy; AUROC = 0.5 corresponds to chance under this scalar summary.

In all reported plots, “higher privacy risk” corresponds to *higher* attack AUROC. This directional convention is critical for interpretation because utility metrics move in the opposite direction (higher is better), making joint utility–privacy analysis inherently bi-objective.

Remark 1. Our evaluation is transductive on a fixed graph: the same G supports training and inference masks. This matches common semi-supervised node classification setups and should not be interpreted as full graph-inclusion membership. MIAs remain non-trivial because posteriors on supervised versus held-out nodes need not be exchangeable under message passing.

C. Threat-model variants outside scope

Our benchmark deliberately targets score-based black-box membership inference. We do not evaluate white-box MIAs over embeddings or gradients, structure-modification attacks, or federated graph-learning threat models [7]. These settings are important extensions but require different access assumptions and evaluation protocols than the API-style probability-access setting studied here.

VI. DATASETS AND SYNTHETIC GENERATION PROTOCOL

A. Design rationale

To isolate homophily and density we generate synthetic graphs with fixed (n, d, C) while systematically varying (i) the fraction of edges connecting same-label endpoints and (ii) the

TABLE III
SYNTHETIC GRAPH GENERATOR: FIXED AND VARIED FACTORS.

Factor	Levels	Role
Homophily target	low (≈ 0.3), high (≈ 0.8)	Same-label edge fraction
Density ρ	0.005, 0.015, 0.04	Fraction of possible edges
Nodes n	400	Fixed order
Features d	50	Input dimensionality
Classes C	5	Label cardinality
Split	50% train / 50% test	Resampled per seed

TABLE IV
APPROXIMATE UNDIRECTED EDGE BUDGET BY DENSITY AT $n = 400$
(BEFORE DUPLICATE REJECTION).

Density label	ρ	Edges (approx.)
Sparse	0.005	~ 400
Medium	0.015	~ 1200
Dense	0.040	~ 3200

total edge budget relative to the complete graph [7]. Observed homophily after generation is reported as

$$h(G) = \frac{1}{|E|} \sum_{(i,j) \in E} \mathbb{1}\{y_i = y_j\}, \quad (2)$$

and undirected density as $|E|/\binom{n}{2}$.

B. Procedural specification

For each seed, labels are sampled uniformly over classes. Class means $\mu_c \in \mathbb{R}^d$ are drawn once per graph; node features are $x_v = \mu_{y_v} + \epsilon_v$ with $\epsilon_v \sim \mathcal{N}(0, \sigma^2 I)$ and fixed σ . Target edge counts scale as $\rho \binom{n}{2}$ with $\rho \in \{0.005, 0.015, 0.04\}$ for sparse, medium, and dense regimes at $n=400$, yielding approximately 400, 1200, and 3200 undirected edges before deduplication effects. A prescribed fraction of edges is allocated to within-class pairs versus cross-class pairs to target low homophily (≈ 0.3) or high homophily (≈ 0.8). Table III summarizes; Table IV gives approximate realized scales. Fig. 1 provides a block diagram of the generator aligned with the implementation: stochastic labels induce clustered features; edge loops enforce ρ and h_{tgt} ; masks finalize the semi-supervised dataset object consumed by trainers.

C. Edge-sampling implementation details

Let $N_{\text{tgt}}(\rho) = \lfloor \rho n(n-1)/2 \rfloor$ be the undirected edge budget. We allocate $\lfloor h_{\text{tgt}} N_{\text{tgt}} \rfloor$ edges to within-class proposals and the remainder to cross-class proposals, where $h_{\text{tgt}} \in \{0.3, 0.8\}$. Candidate pairs (i, j) are sampled under label constraints, duplicates are rejected with a hash set, and accepted pairs are inserted in both orientations for PyG edge-index compatibility. If no edge survives proposal caps, we inject a minimal two-node edge to avoid degenerate empty-graph tensors. Because rejection sampling enforces uniqueness, realized $h(G)$ and density can deviate slightly from targets and are therefore measured and reported per run.

D. Citation benchmarks

Planetoid Cora and Citeseer statistics appear in Table VI. We evaluate Cora and Citeseer with the same model, attack, defense, and seed protocol, while using them as real citation benchmarks rather than controlled structural regimes (see Figs. 3 and 11).

VII. PREDICTIVE MODELS

This section specifies every hypothesis class used in RQ1–RQ4. Baselines intentionally *ignore* adjacency so that differences in membership leakage isolate graph-dependent mechanisms versus feature-only overfitting.

A. Multinomial logistic regression

Multinomial logistic regression implements a linear softmax classifier directly on the node features. Specifically, given learned parameters $W \in \mathbb{R}^{C \times d}$ and $b \in \mathbb{R}^C$, the model computes the softmax transform $\hat{p}_v = \text{softmax}(Wx_v + b)$ over the node features x_v . Training minimizes the empirical cross-entropy loss $\mathcal{L}_{\text{LR}} = -\sum_{v \in \mathcal{T}} \log \hat{p}_{v, y_v}$ without an ℓ_2 penalty term in the objective function beyond the defaults for the optimization solver. The solver uses either L-BFGS-B or coordinate descent, depending on the dimensionality of the feature space, as implemented in the scikit-learn library with tightened convergence tolerances.

Role in the study. Since logistic regression is a convex model with a readily interpretable parameterization, its AUROC provides a baseline against which to compare the performance of the GCN. If the AUROC for the attacked model is substantially higher for a logistic regression model than for the GCN, it indicates that even without the use of message passing and nonlinear transformation of the node features, the model still exhibits a membership signal. Therefore, the performance of the GCN can be evaluated in relation to this baseline result.

B. Multilayer perceptron

The multilayer perceptron (MLP) applies compositional nonlinearities to node features while still discarding A . Hidden representations follow

$$\begin{aligned} h_v^{(1)} &= \text{ReLU}(W^{(1)}x_v + b^{(1)}), \\ h_v^{(2)} &= \text{ReLU}(W^{(2)}h_v^{(1)} + b^{(2)}), \\ z_v &= W^{(3)}h_v^{(2)} + b^{(3)}. \end{aligned} \quad (3)$$

with hidden widths 64 and 32, matching the GCN hidden width in the first layer for rough parameter-count comparability. Training minimizes the same cross-entropy on $\mathcal{T}$ using backpropagation as implemented in scikit-learn’s MLPClassifier.

Role in the study. The MLP tests whether deeper nonlinear feature extraction—without structure—recovers the qualitative leakage trends of GCNs. If GCN AUROC greatly exceeds MLP AUROC under heterophily, graph propagation likely contributes memorization-like confidence gaps beyond universal approximation on X alone.

TABLE V

SYNTHETIC EVALUATION REGIMES: NAMING CONVENTION, STRUCTURAL TARGETS, AND GRAPH-SCALE STATISTICS AT $n=400$ (UNDIRECTED). OBSERVED $h(G)$ AND $|E|$ VARY SLIGHTLY PER SEED DUE TO REJECTION SAMPLING; TYPICAL RANGES ALIGN WITH TARGET BANDS.

Dataset ID	Homophily band	ρ	$ E $ (approx.)	Purpose
synthetic_high_sparse	high (~ 0.8)	0.005	~ 400	Sparse assortative
synthetic_high_medium	high	0.015	~ 1200	Medium assortative
synthetic_high_dense	high	0.040	~ 3200	Dense assortative
synthetic_low_sparse	low (~ 0.3)	0.005	~ 400	Sparse heterophilic
synthetic_low_medium	low	0.015	~ 1200	Medium heterophilic
synthetic_low_dense	low	0.040	~ 3200	Dense heterophilic

Common to all: $d=50$, $C=5$, labels sampled iid uniformly, Gaussian blob features.

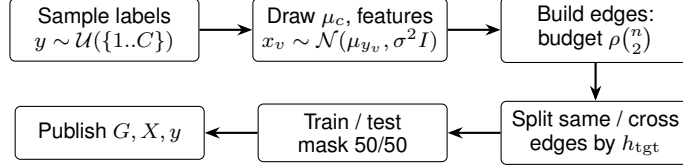


Fig. 1. Controlled synthetic pipeline: labels drive feature clusters; edges satisfy density ρ and homophily target h_{tgt} ; masks split nodes for semi-supervised training.

TABLE VI
PLANETOID DATASETS AVAILABLE IN THE IMPLEMENTATION.

Dataset	Nodes	Edges	Features	Classes
Cora	2,708	5,429	1,433	7
Citeseer	3,327	4,732	3,703	6

C. Graph convolutional network

We instantiate the spectral message-passing stack in (1), with standard self-loop normalization as implemented by PyTorch Geometric’s GCNConv. Each row of $H^{(1)}$ mixes a node’s features with those of its neighbors via $\tilde{A}$, encouraging representations to agree along edges; the second layer repeats mixing on intermediate activations. Dropout with probability 0.5 is applied to $H^{(1)}$ during training. Weight matrices have shapes $W^{(0)} \in \mathbb{R}^{d \times 64}$ and $W^{(1)} \in \mathbb{R}^{64 \times C}$. Parameters are learned with Adam on the node-level cross-entropy restricted to training masks.

Inductive capacity and privacy. Full-batch GCN is transductive on G : parameters interact with the entire adjacency during training. Consequently, memorizing localized topology quirks can sharpen posteriors on training nodes relative to held-out nodes even under modest global accuracy gaps—precisely the asymmetry membership attackers exploit [3]. Defenses in Section IX perturb optimization paths through A or labels without necessarily shrinking parameter norms.

D. GraphSAGE with mean pooling

GraphSAGE learns differentiable aggregation functions over neighborhoods [9]. With mean pooling, each layer computes neighbor averages of transformed features before concatenation or summation with self-features depending on operator variants; our PyTorch Geometric implementation uses the GraphSAGE convolution operator with mean aggregation in

full-batch mode so every neighbor participates without subsampling.

Contrast with GCN. Whereas spectral GCN layers correspond to polynomial filters on the normalized adjacency, GraphSAGE’s aggregation is parameterized distinctly per layer; empirical studies often report improved robustness under heterophily when paired with appropriate filters. In our benchmark, GraphSAGE maintains competitive accuracy with materially different AUROC than GCN on several low-homophily regimes, motivating architecture-aware privacy decisions rather than treating “GNN” as monolithic.

VIII. MEMBERSHIP INFERENCE ATTACKS

A. Score features and attack input representation

Given probability vector $\hat{p}_v$ and true label y_v , we form a fixed, four-dimensional vector $\phi(\hat{p}_v, y_v) \in \mathbb{R}^4$ shared across all attack families below:

$$\phi_1 = \max_c \hat{p}_{v,c}, \quad (4)$$

$$\phi_2 = \hat{p}_{v,y_v}, \quad (5)$$

$$\phi_3 = - \sum_{c=1}^C \hat{p}_{v,c} \log(\hat{p}_{v,c} + \varepsilon), \quad (6)$$

$$\phi_4 = -(1 - \phi_1) \log(\phi_1 + \varepsilon), \quad (7)$$

with small ε for numerical stability. Here ϕ_1 captures peak confidence, ϕ_2 the model’s assigned probability to the *true* class (often elevated for training points under overfitting), ϕ_3 is Shannon entropy of the predictive distribution, and ϕ_4 supplies a margin-style transform that correlates with certainty when mass concentrates on a single class [2]. Restricting to four coordinates keeps the attack lightweight and reproducible while retaining expressive power for linear meta-classifiers.

B. Confidence-based attack with held-out attack validation

Let $\mathcal{D}_{\text{atk}} = \{(\phi(\hat{p}_v, y_v), \mathbb{1}\{v \in \mathcal{T}\}) : v \in \mathcal{T} \cup \mathcal{S}\}$. We shuffle indices, allocate half of the tuples to train a logistic regression attacker and the other half to score AUROC and accuracy. This internal split mitigates trivial overfitting of the attack model to the same nodes it evaluates, though it does not constitute a fully independent graph draw; shadow attacks below partially address distributional shift.

Interpretation. High AUROC implies that linear combinations of ϕ separate member from non-member posteriors. Failure (AUROC ≈ 0.5) suggests posteriors are statistically similar across masks under these features—either because the target generalizes well in confidence space or because the attack representation is too coarse.

C. Threshold attack on true-class confidence

The threshold attack constructs a *single* scalar score per node, namely ϕ_2 . We compute the global median τ across both members and non-members, then treat $\phi_2 > \tau$ as the membership prediction rule; AUROC evaluates how well this score ranks members. This extremely parsimonious adversary is useful as a sanity check: if sophisticated attacks succeed but the threshold attack fails, leakage may hinge on nonlinear combinations of entropy and margin captured only in $\phi_{1,3,4}$.

D. Shadow-model attack and transfer

We instantiate a shadow classifier with identical architecture, optimizer, and defense schedule as the target, trained on an independent synthetic sample (new seed) or on a resampled Planetoid split. Attack training stacks shadow-derived ϕ -vectors with membership labels; evaluation applies the fitted attacker to ϕ extracted from the *target* posteriors [1]. Transfer failure indicates strong distribution shift between shadow and target posteriors, whereas transfer success corroborates confidence-based findings.

E. Calibration error

Let $\text{conf}_v = \max_c \hat{p}_{v,c}$ and $\text{acc}_v = \mathbb{1}\{\arg \max \hat{p}_v = y_v\}$. Partition $[0, 1]$ into B equal-width bins. Expected calibration error (ECE) is

$$\text{ECE} = \sum_{b=1}^B \frac{|V_b|}{|V_{\text{te}}|} |\text{acc}(V_b) - \overline{\text{conf}}(V_b)|, \quad (8)$$

where V_b are test nodes whose confidence falls in bin b , and $\text{acc}(V_b)$, $\overline{\text{conf}}(V_b)$ denote bin averages. We use $B=10$ on test nodes.

IX. LIGHTWEIGHT DEFENSES

Unless noted, defenses apply only to GNN parameters; baselines always train under vanilla cross-entropy without structural modifications. Each mechanism below therefore isolates a distinct inductive bias: stochastic topology (DropEdge), target smoothing (label smoothing), optimization truncation (early stopping), output compression (masking), or permanent edge deletion (sparsification).

A. Vanilla training baseline

The baseline minimizes $\mathcal{L}_0 = -\sum_{v \in \mathcal{T}} \log \hat{p}_{v,y_v}$ using Adam with fixed learning rate and weight decay until the epoch budget or early-stopping rule fires. No edge dropout, smoothing, or masking is applied. This configuration anchors all comparative statements.

B. DropEdge as stochastic graph augmentation

Mechanism. DropEdge [10] treats message-passing edges as a random set that can be subsampled independently at each forward pass during training. Concretely, each training epoch draws a Bernoulli mask on edges with retention probability $1 - r_{\text{de}}$; we set $r_{\text{de}} = 0.3$. Message passing therefore sees $G^{(t)} = (V, E^{(t)})$ where $E^{(t)}$ differs across iterations even though the underlying stored graph remains intact.

Rationale. Random edge deletion acts as structured dropout: it discourages the optimizer from relying on any single edge for lowering loss, potentially smoothing decision boundaries and reducing overfitting to idiosyncratic neighborhoods—effects hypothesized to correlate with lower membership leakage when memorization drives confidence gaps.

Inference. At test time we revert to the full edge set E except when confidence masking post-processes logits. Hence DropEdge perturbs optimization dynamics without permanently damaging connectivity information available at deployment.

C. Label smoothing as KL training

Mechanism. Rather than one-hot vectors e_{y_v} , we minimize $\text{KL}(\tilde{q}_v \| \text{softmax}(z_v))$ where $\tilde{q}_{v,y_v} = 1 - \alpha + \alpha/C$ and $\tilde{q}_{v,c} = \alpha/C$ for $c \neq y_v$, with $\alpha = 0.1$. Equivalently, this penalizes extremal logits even when the classifier would confidently separate training points.

Rationale. Smoothing is often motivated by calibration and adversarial robustness; for membership inference, the hypothesis is that softer targets shrink the gap between member and non-member posteriors [3].

Empirical caveat. Our experiments show that smoothing can *increase* attack AUROC and ECE on graph models, implying that redistributing mass without aligning train-test exchangeability may amplify rather than attenuate score-based leakage.

D. Early stopping based on training loss trajectories

Mechanism. Because no validation mask is reserved in the minimal pipeline, early stopping monitors the *training* cross-entropy with patience $p = 15$: if for p consecutive epochs the loss fails to improve by at least $\delta = 10^{-4}$ relative to the best epoch t^* , weights roll back to the checkpoint at t^* and optimization halts.

Rationale. The intended effect mirrors classical regularization: halt before parameters encode fine-grained noise. This interacts subtly with graph losses because neighborhood coupling may cause loss plateaus even when membership-relevant confidence patterns continue evolving.

TABLE VII
DEFENSE HYPERPARAMETERS (FIXED ACROSS THE GRID).

Defense	Value
DropEdge	$r_{de} = 0.3$
Label smoothing	$\alpha = 0.1$
Early stopping	patience $p = 15$, $\delta = 10^{-4}$
Confidence masking	top- k , $k = 2$
Edge sparsification	$r_{es} = 0.2$

E. Confidence masking at inference time

Mechanism. After computing softmax outputs, we retain the top- k predicted class probabilities with $k = 2$, set all other probabilities to zero, and renormalize the retained mass. Shadow models undergo the same post-processing so attack transfer reflects realistic masked APIs.

Rationale. Masking hides tail probabilities from clients, modeling settings where only top-ranked classes are released [11]. Entropy-based MIAs may degrade if ϕ_3 loses sensitivity; conversely, if members concentrate remaining mass differently than non-members, AUROC can persist.

Sensitivity sketch. Confidence masking induces a piecewise gate on logits before softmax. If $\mathcal{K}$ is the retained top- k index set, gradients of $\hat{p}$ with respect to suppressed logits are effectively zeroed, while active coordinates retain the standard softmax Jacobian. This explains why attacks relying on tail-probability behavior may weaken, yet member/non-member gaps can remain through top-class statistics.

F. Edge sparsification before optimization

Mechanism. Unlike DropEdge, sparsification applies an edit to E only once before training. Each undirected edge pair has a probability $r_{es} = 0.2$ of being deleted; the resulting graph $\tilde{E} \subset E$ is used for training and inference. Deleting both stored directions of an edge pair ensures that undirected semantics are respected.

Rationale. By removing edges from the stored graph, the receptive field of the network is reduced. Such an operation has mixed results in that using the same sparsified graph for training, inference, and shadow evaluation does not provide a gain in privacy for GCN on low-homophily medium-density graphs.

Comparison to DropEdge. DropEdge samples the edges of the stored graph E at each step of training; sparsification does not offer a re-sampling mechanism. Hence, the two methods are not interchangeable.

X. EXPERIMENTAL PROTOCOL

A. Optimization

Adam with learning rate $\eta = 0.01$, weight decay 5×10^{-4} , maximum $T=50$ epochs, PyTorch CPU backend in reported runs. Defense hyperparameters match Table VII.

B. Replication design and configuration management

Seeds $\mathcal{S} = \{42, 123, 456, 789, 1024\}$ index independent train-test permutations and synthetic instantiations. Each tuple (dataset, model, defense, seed) yields one CSV row of metrics.

Hyperparameters such as learning rate, epoch cap, and defense rates live in a YAML file so that reproducing or extending the study does not require modifying Python modules beyond plugin hooks; this separation mirrors best practices for ML reproducibility in security-sensitive evaluations.

This paper reports the `experiment_config_paper.yaml` grid: Cora, Citeseer, six synthetic regimes, four model families, five random seeds, five lightweight GNN defenses plus no defense, and confidence/threshold/shadow attacks. Other repository components (e.g., large-graph, LiRA, and DP-SGD hooks) are treated as future extensions and are not part of the reported claims.

C. Implementation details and attack-split conventions

Experiments run in Python with PyTorch and PyTorch Geometric for graph training and Planetoid loading, scikit-learn for feature-only baselines and attack classifiers, and pandas for aggregation. For synthetic graphs ($n=400$), CPU execution is sufficient for full sweeps. The confidence attacker uses a 50/50 shuffle split over member/non-member tuples for internal attack train/eval partitioning. Shadow models for synthetic settings regenerate data using seed $s+999$ to reduce overlap with target realizations while preserving the same generator family.

D. Reproducibility checklist

The YAML configuration enumerates datasets $\times$ models $\times$ defenses $\times$ seeds; running the main driver produces row-level CSVs, then summary and significance tables. Figure scripts consume these stored CSV artifacts directly. Controlled randomness enters through split permutations, synthetic edge sampling, DropEdge masks, and attack subsplits; fixed seeds make these paths repeatable.

E. Statistical testing

For defense d versus none, let $a_s^{(d)}$ and $a_s^{(0)}$ denote paired confidence-attack AUROC at seed s . We report mean differences and two-sided paired t -tests with test statistic

$$t = \frac{\bar{\Delta}}{s_{\Delta}/\sqrt{|\mathcal{S}|}}, \quad \Delta_s = a_s^{(d)} - a_s^{(0)}, \quad (9)$$

with $|\mathcal{S}|=5$. We also compute percentile bootstrap confidence intervals over seeds for metric means. Normality is not guaranteed for AUROC differences at tiny sample sizes, and no multiple-comparison correction is applied; we therefore interpret p -values as exploratory indicators rather than definitive causal claims.

F. Computational complexity

Per trial, target training costs $\mathcal{O}(T|E|d_h)$ for sparse matrix multiplies in two GCN layers with hidden width $d_h=64$, dominated by graph convolution when $|E|$ is $\mathcal{O}(\rho n^2)$ at medium density. Shadow training duplicates this cost. Attack training on $\mathcal{O}(n)$ nodes with four features is negligible versus graph forward passes. Overall wall-clock scales roughly linearly in $|E|$ for fixed T .

Algorithm 1 Single trial (dataset, model, defense, seed)**Require:** Graph family, model class, defense configuration, seed s

- 1: Instantiate or load data; apply train-test split from s
- 2: Train target model with defense on training mask
- 3: Compute posteriors on training and test nodes
- 4: Train shadow model on independent split or regenerated graph
- 5: Compute ϕ -features; run confidence, threshold, shadow attacks
- 6: Evaluate accuracy, F1, AUROC, ECE on test mask
- 7: **return** Metric vector (stored per row)

G. Algorithmic summary

Algorithm 1 sketches one experimental trial.

H. Pipeline diagram and methodological structure

Fig. 2 orders the benchmark stages and should be read together with Fig. 1: the latter explains *how* graphs arrive in the repository, while the former explains *how* models, shadows, and attackers consume them. Tables III–V catalogue input statistics; Section VII fixes hypothesis classes; Section VIII formalizes threats; Section IX enumerates mitigations.

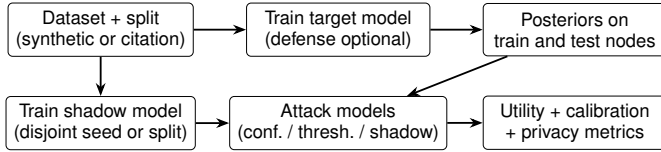


Fig. 2. End-to-end benchmark: target training, posterior extraction, shadow training, attack evaluation, and reporting.

XI. EMPIRICAL RESULTS

Unless stated otherwise, numerical summaries aggregate five seeds per cell using mean and standard deviation in plots and tables. Section VI and Table V identify each synthetic regime; Section VIII defines attack scores whose AUROC we report.

A. Comparative vulnerability

Fig. 3 summarizes AUROC without defense. Under high homophily, attack AUROC clusters near 0.5, indicating limited separability between member and non-member score profiles. Under low homophily and sparsity, GCN rises to ≈ 0.59 while GraphSAGE and non-graph models remain near 0.51–0.53. This gap is practically meaningful because it appears under the same attack protocol and data family, suggesting that leakage is not a generic consequence of using graphs but an interaction between architecture and structural regime. Fig. 4 complements this view with a compact heatmap across models and synthetic settings.

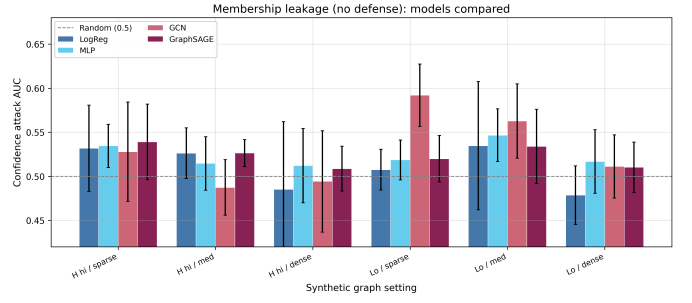


Fig. 3. Attack AUROC without defense across model families.

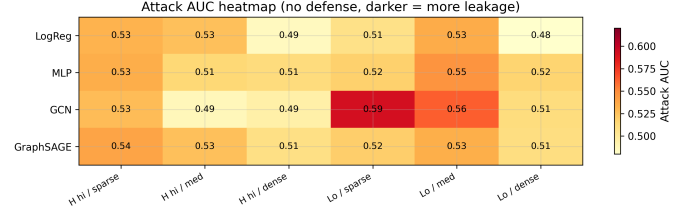


Fig. 4. No-defense attack AUROC heatmap across models and synthetic regimes.

B. Citation benchmark behavior

Cora and Citeseer complicate any simple claim that graph models leak more than feature-only baselines. With no defense, Cora confidence-attack AUROC is 0.735 for LogReg, 0.772 for MLP, 0.554 for GCN, and 0.635 for GraphSAGE. Citeseer shows the same qualitative pattern: 0.730 for LogReg, 0.830 for MLP, 0.632 for GCN, and 0.720 for GraphSAGE. These results suggest that feature separability and posterior confidence can dominate graph-specific effects on citation datasets, whereas controlled synthetic regimes isolate when graph structure itself makes GCN more vulnerable.

C. Structural mediation

Attack AUROC for GCN rises as homophily drops and tends to increase when ρ decreases within the low-homophily family. GraphSAGE frequently achieves high predictive accuracy with AUROC still near chance, underscoring that accuracy and membership distinguishability need not align. Put differently, utility parity does not imply privacy parity: two models can be equally accurate while exposing different membership signals through their posterior geometry.

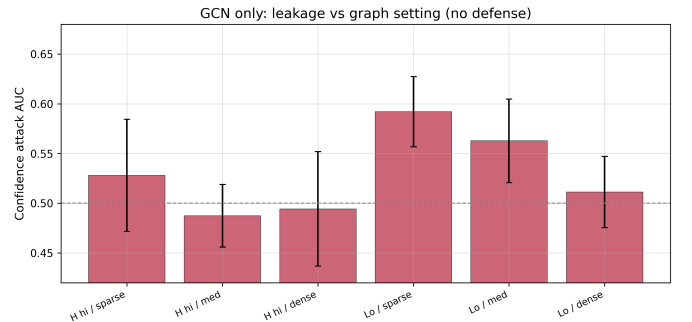


Fig. 5. GCN without defense across synthetic regimes.

D. Defense interactions

Label smoothing increases AUROC in multiple cells; paired tests identify degradation for GraphSAGE on low-homophily sparse graphs at $\alpha=0.1$ (+0.094 AUROC, $p = 0.0421$), for Cora GCN (+0.039 AUROC, $p = 0.0039$), and for Citeseer GraphSAGE (+0.067 AUROC, $p = 3.1 \times 10^{-6}$). Its calibration effects are dataset-dependent: it reduces ECE in some settings but increases ECE or creates more membership-separable posterior patterns in others. After correcting edge sparsification to use the same undirected-thinned graph for training, inference, and shadow evaluation, the earlier low-homophily medium-density GCN privacy improvement does not persist (0.5627 to 0.5638 AUROC, $\Delta = +0.0011$, $p = 0.9684$; bootstrap 95% CIs [0.530, 0.596] vs. [0.526, 0.619]). DropEdge, early stopping, masking, and sparsification therefore exhibit regime-dependent effects rather than universal privacy improvements.

TABLE VIII

ILLUSTRATIVE PAIRED-TEST HIGHLIGHTS FOR CONFIDENCE-ATTACK AUROC. POSITIVE Δ MEANS HIGHER LEAKAGE; p -VALUES ARE EXPLORATORY BECAUSE MANY COMPARISONS ARE EVALUATED.

Setting / model	Defense	Δ AUC	p
Low-medium / GCN	Edge sparsif.	+0.001	0.9684
Low-sparse / SAGE	Label smooth.	+0.094	0.0421
Cora / GCN	Label smooth.	+0.039	0.0039
Citeseer / SAGE	Label smooth.	+0.067	3.1×10^{-6}

TABLE IX

MEAN CONFIDENCE-ATTACK AUROC WITHOUT DEFENSE ($|S|=5$).

Setting	LogReg	MLP	GCN	SAGE
High homophily, dense	0.485	0.536	0.494	0.509
High homophily, sparse	0.532	0.504	0.528	0.539
Low homophily, medium	0.535	0.561	0.563	0.534
Low homophily, sparse	0.508	0.506	0.592	0.520

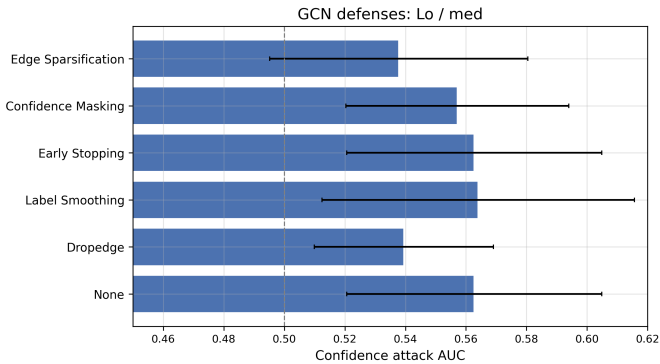


Fig. 6. GCN defenses on low-homophily medium-density graphs.

E. Utility-privacy and structure-aware trends

Figs. 7 and 8 visualize the utility-privacy frontier and measured-homophily trend, highlighting where high utility coexists with low attack AUROC. Figs. 9 and 10 complement this view with density-wise leakage trends and a defense effectiveness panel for the citation benchmarks.

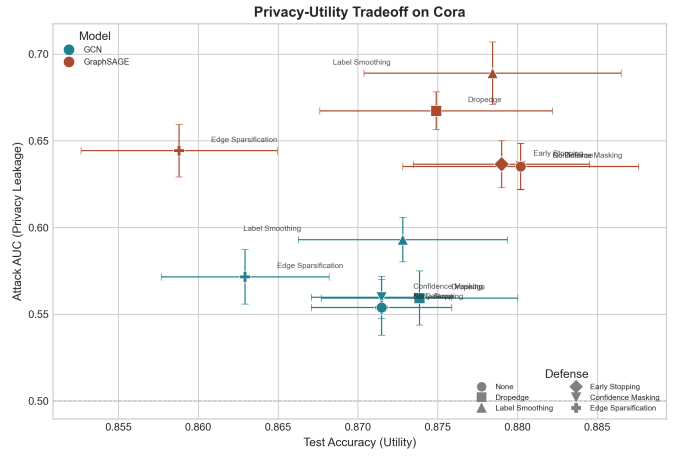


Fig. 7. Privacy-utility scatter across GNN defenses.

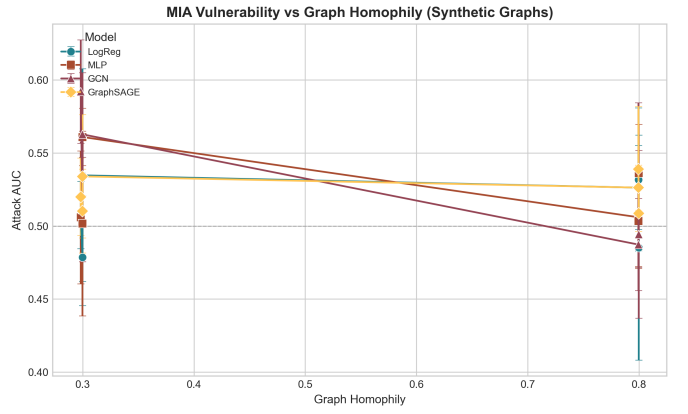


Fig. 8. Attack AUROC versus measured homophily in synthetic regimes.

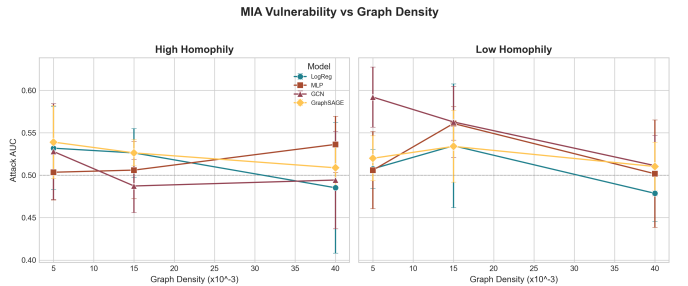


Fig. 9. Density-wise leakage trends across homophily regimes.

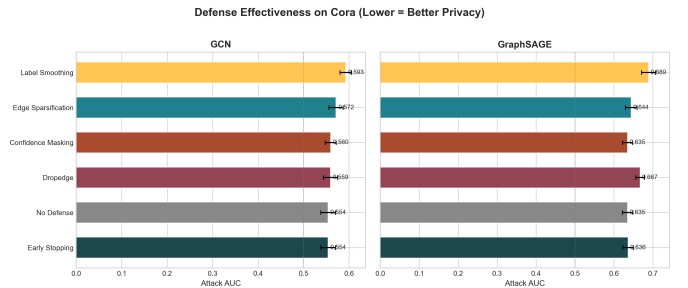


Fig. 10. Defense effectiveness ranking for citation benchmarks.

F. Citation-oriented and comprehensive views

Fig. 11 provides a citation-based comparison panel, while Fig. 12 gives a comprehensive heatmap that summarizes model/setting behavior at a glance.

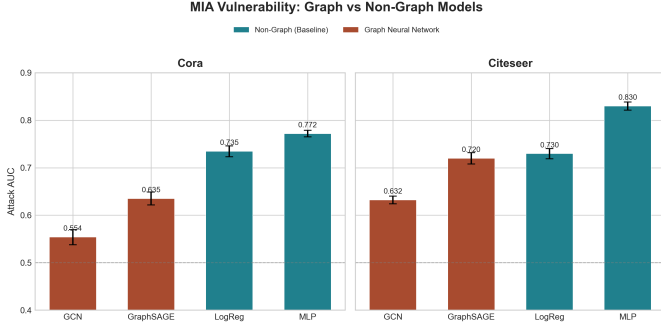


Fig. 11. Citation benchmark comparison across model families.

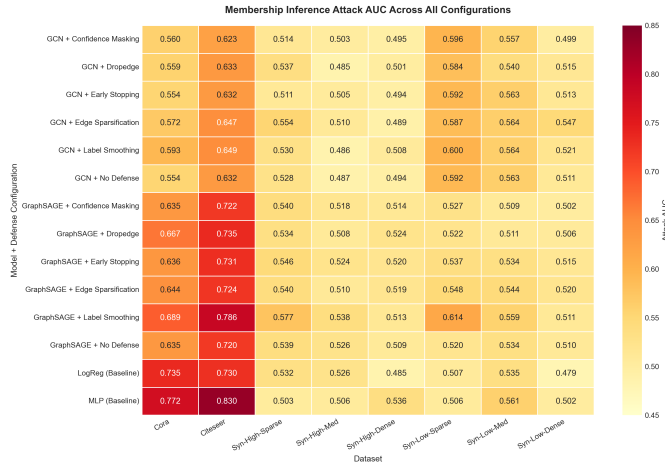


Fig. 12. Comprehensive attack AUROC heatmap across datasets, models, and defenses.

XII. DISCUSSION

Mechanistic interpretation. In high-homophily settings, neighbors of a supervised node tend to have similar class predictions to the supervised node itself, leading to similar posterior distributions on supervised versus held-out nodes under black-box score tests. Under low-homophily settings, the aggregation operation of a GCN tends to create a loss and confidence signal that is separable between training and test node masks [3], [5], [25], [26]. Within the citation datasets, however, models relying solely upon the node features of a graph tend to exhibit a greater degree of membership leakage than GNN models, indicating that the separability of the feature representation may be the main factor influencing the leakage of membership information in these settings. Thus, these results indicate that the main factor driving the leakage of membership is the difference in the shape of the model confidence distributions for supervised versus held-out nodes.

Defense pitfalls. Label smoothing has been shown to work well in smoothing the posterior distributions for supervised

nodes, but with no guarantee of flatter posteriors for supervised versus held-out nodes in graph settings; in some datasets, label smoothing can even increase the AUROC of the attacker [11]. Similarly, permanent edge sparsification can remove structural pathways between supervised and held-out nodes prior to training, but the resulting models have shown mixed results in enforcing membership privacy. Thus, calibration and topology-based defenses should not be assumed to have any privacy-preserving properties without evaluating their impact upon attack success.

Limitations. The synthetic graphs used in this paper do not include models of community structure, heterophily, or feature noise. The node labels are sampled iid uniformly over the number of classes, and the node features are clustered Gaussian blobs grouped according to their class labels; thus, feature-only methods can achieve high accuracy. The synthetic graph setting enables studying homophily and density effects while excluding the influence of feature noise. However, it does not exhaustively consider feature-noise regimes where graph structure alone may be more important for node classification. In this work, only the lightweight score-based MIAs are evaluated. More strict evaluations of label-only [11], likelihood-ratio [4], and low false positive rate (FPR) auditing methods can offer further insight into the efficacy of the MIAs. Five random seeds are used to generate the results. More seeds would provide better statistical power in the evaluations. The significance of the difference in accuracy between two methods is evaluated using the paired t -test. However, because there are so many defenses and regimes evaluated in this work, the p -values from the t -tests should only be considered exploratory. The defenses proposed in this work are lightweight and practical; however, formally private training methods such as differentially private (DP) GNNs, e.g., GAP [12], are the best methods to employ when meeting regulatory privacy requirements.

A. Practitioner guidelines

Deployers should (i) define the membership predicate precisely before auditing; (ii) record confidence histograms and ECE alongside accuracy when evaluating privacy interventions; (iii) avoid interpreting label smoothing as a privacy mechanism absent attack evaluation on graph data; (iv) treat structural sparsification as a setting-dependent heuristic rather than a guaranteed privacy defense; and (v) plan for adversaries stronger than linear ϕ -attacks when stakes are high [4].

XIII. CONCLUSION

This paper studied when graph neural networks leak information about which nodes were used for supervised training, and whether simple defenses can reduce that leakage. The main conclusion is that membership privacy in graph learning is not determined by the model alone. It depends on the interaction between the graph structure, the model architecture, the strength of the node features, the confidence behavior of the model, and the defense being applied. In controlled synthetic graphs, GCNs showed the highest leakage when the

graph structure was unfavorable, especially in low-homophily regimes where connected nodes often belonged to different classes. In citation benchmarks, however, feature-only models could leak more when the node features were already highly separable. These results show that there is no single rule such as “GNNs always leak more” or “graph structure always protects privacy.”

The defense results also show that lightweight privacy mechanisms must be evaluated carefully. Label smoothing, which is often expected to reduce overconfidence, frequently increased attack AUROC in our experiments. Edge sparsification produced mixed results after being implemented as permanent undirected edge thinning for both training and inference. Overall, the most important lesson is that privacy risk in graph learning is *structurally contingent*: it must be measured under the specific graph, model, and deployment setting being used. Future work should expand this study with larger replications, stronger likelihood-ratio attacks [4], low-false-positive-rate auditing, feature-difficulty ablations, and differentially private GNN baselines [12]. These extensions would support broader model-risk and deployment auditing practices for graph-based machine learning systems [14], [22].

CODE AVAILABILITY

The code and experiment configuration files used in this study are publicly available at <https://github.com/krishharma/privacy-gnn>. The repository includes YAML configuration files, PyTorch and scikit-learn implementations, CSV aggregation scripts, and matplotlib figure-generation scripts needed to reproduce the reported tables and plots.

ACKNOWLEDGMENT

Research conducted in the Data Science Laboratory (DiSC), University of Nevada, Las Vegas.

REFERENCES

- [1] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, “Membership inference attacks against machine learning models,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2017, pp. 3–18.
- [2] A. Salem, Y. Zhang, M. Humbert, P. Berrang, M. Fritz, and M. Backes, “ML-Leaks: Model and data independent membership inference attacks and defenses on machine learning models,” in *Proc. Netw. Distrib. Syst. Security Symp. (NDSS)*, 2019.
- [3] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, “Privacy risk in machine learning: Analyzing the connection to overfitting,” in *Proc. IEEE Comput. Security Found. Symp. (CSF)*, 2018, pp. 268–282.
- [4] N. Carlini, S. Chien, M. Nasr, A. Song, T. Terzis, and F. Tramer, “Membership inference attacks from first principles,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2022, pp. 1897–1914.
- [5] I. E. Olatunji, W. Nejdl, and M. Khosla, “Membership inference attack on graph neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.
- [6] Y. Wu, D. Li, B. Chen, and L. Shen, “Adapting membership inference attacks to GNN for graph classification,” in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2021, pp. 1429–1434.
- [7] Y. Zhang, Y. Zhao, Z. Li, X. Cheng, Y. Wang, O. Kotevska, P. S. Yu, and T. Derr, “A survey on privacy in graph neural networks: Attacks, preservation, and applications,” *arXiv preprint arXiv:2308.16375*, 2023.
- [8] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.
- [9] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 1024–1034.
- [10] Y. Rong, W. Huang, T. Xu, and J. Huang, “DropEdge: Towards deep graph convolutional networks on node classification,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [11] C. A. Choquette-Choo, F. Tramer, N. Carlini, and N. Papernot, “Label-only membership inference attacks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2021, pp. 1964–1974.
- [12] S. Sajadmanesh, J. G. Castillo, A. Naseri, M. Maengel, J. Boedecker, and B. H. F. Gomes, “GAP: Differentially private graph neural networks with aggregation perturbation,” in *Proc. USENIX Security Symp.*, 2023, pp. 3229–3246.
- [13] M. Fredrikson, S. Jha, and T. Ristenpart, “Model inversion attacks that exploit confidence information and basic countermeasures,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2015, pp. 1322–1333.
- [14] F. Tramèr, F. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, “Stealing machine learning models via prediction APIs,” in *Proc. USENIX Security Symp.*, 2016, pp. 601–618.
- [15] C. Song, T. Ristenpart, and V. Shmatikov, “Machine learning models that remember too much,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2017, pp. 587–601.
- [16] M. Nasr, R. Shokri, and A. Houmansadr, “Comprehensive privacy analysis of deep learning: Stand-alone and federated learning under passive and active white-box inference attacks,” in *Proc. IEEE Symp. Security Privacy (S&P)*, 2019, pp. 739–753.
- [17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 1321–1330.
- [18] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2005, pp. 625–632.
- [19] C. Dwork, “Differential privacy,” in *Proc. Int. Colloq. Automata, Languages and Programming (ICALP)*, 2006, pp. 1–12.
- [20] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, “Deep learning with differential privacy,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Security (CCS)*, 2016, pp. 308–318.
- [21] I. Mironov, “Rényi differential privacy,” in *Proc. IEEE Comput. Security Found. Symp. (CSF)*, 2017, pp. 263–275.
- [22] B. Jayaraman and D. Evans, “Evaluating differentially private machine learning in practice,” in *Proc. USENIX Security Symp.*, 2019, pp. 1895–1912.
- [23] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [24] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2019.
- [25] H. Pei, B. Wei, K. C.-C. Chang, Y. Lei, and B. Yang, “Geom-GCN: Geometric graph convolutional networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2020.
- [26] J. Zhu, Y. Yan, L. Zhao, Y. Heimann, L. Akoglu, and D. Koutra, “Beyond homophily in graph neural networks: Current limitations and effective designs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [27] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, “Open graph benchmark: Datasets for machine learning on graphs,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020.
- [28] X. Zhu, Z. Ghahramani, and J. D. Lafferty, “Semi-supervised learning using Gaussian fields and harmonic functions,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2003.
- [29] V. Duddu, A. Boutet, and V. Shejwalkar, “Quantifying privacy leakage in graph embedding,” in *Proc. ACM Int. Conf. Web Search Data Mining (WSDM)*, 2020, pp. 76–84.
- [30] M. Hay, G. Miklau, D. Jensen, P. Weis, and S. Srivastava, “Resisting structural re-identification in anonymized social networks,” *Proc. VLDB Endowment*, vol. 1, no. 1, pp. 102–114, 2008.